Relatório de Auditoria de Segurança - Trampô

Auditoria estática orientada a cinco categorias de risco

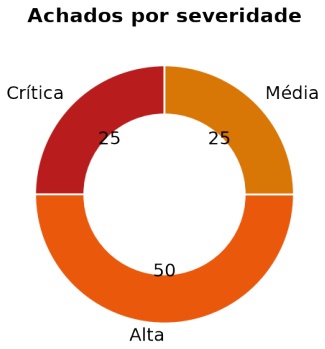
Data: 29/10/2026
Escopo: API Express, React/Vite, React Native/Expo, PostgreSQL, Docker Compose, Render, Vercel, scripts, docs, histórico Git e bundle web.

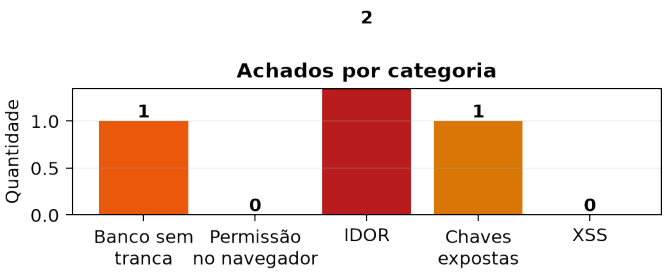
Nota metodológica: o isolamento foi mapeado para filtros manuais de ownership na API; gates de papel foram cruzados com handlers Express; todos os handlers foram listados e lidos; segredos foram buscados em código, configs, docs, histórico e bundle; XSS foi mapeado para sinks React/HTML reais.

Resumo executivo

Foram confirmados 4 achados acionáveis: 1 crítico, 2 alto e 1 médio. Não houve achado verificável nas categorias de permissão definida no navegador e XSS.

Crítica	Alta	Média	Baixa	Pontos fortes
1	2	1	0	6





Stack e modelo de segurança

Backend próprio Express com consultas SQL parametrizadas por pg; autenticação por JWT e bcrypt; frontend React/Vite; app React Native/Expo; PostgreSQL; Docker Compose local; Render e Vercel. Não existe RLS, ORM, middleware de tenant ou workspace: o isolamento depende de filtros manuais por user.id e relações SQL.

Pontos fortes

Stack e auth: Node.js/Express, PostgreSQL via pg, React/Vite, React Native/Expo, JWT próprio com jsonwebtoken e bcrypt. Não há ORM, RLS, Helm ou Terraform.

Autenticação: jwt.verify valida o token e userFrom carrega o usuário por users.id (server/index.js:146-155). Login compara senha com bcrypt (server/index.js:238-245).

Admin: O update de users exige adminOnly (server/index.js:69, 611-615) e leituras com scope=admin exigem isAdmin (server/index.js:576-578).

Fluxos críticos: Aceite, check-in, conclusão e desbloqueio de chat usam rotas dedicadas, verificam participantes e transações PostgreSQL.

XSS: A busca encontrou apenas dangerouslySetInnerHTML em src/components/ui/chart.tsx:69-80, usado para CSS gerado por ChartConfig estático. Não foi encontrado dado de usuário entrando nesse sink, nem eval/new Function/v-html/markdown sem sanitização.

Histórico e bundle: .env não está versionado; a busca no histórico encontrou os arquivos de configuração e defaults auditados, sem chave externa adicional distinta. O bundle é Vite e não contém uma chave de provedor identificada nesta revisão.

Pontos fracos centrais

O CRUD genérico ainda é permissivo por padrão para atualizações não declaradas; leituras de tabelas classificadas como públicas retornam SELECT *; defaults funcionais de banco, JWT e contas demo estão documentados/versionados.

Achados detalhados

Severidade	Arquivo:linha	Descrição
Alta	server/index.js:71; 579-608	**POST de preferências aceita user_id controlado pelo cliente** A policy de preferences habilita insert, mas o bloco de validações de POST cobre apenas jobs, applications, conversations, messages e reviews. O INSERT usa todas as colunas enviadas no body, incluindo user_id, sem substituí-lo pelo usuário autenticado.
Crítica	server/index.js:611-639	**PATCH genérico altera objetos sem validação de posse** A rota /api/data/:table permite PATCH para qualquer tabela cujo policy.update seja undefined. O código valida apenas a presença de eq.* e, para tabelas sem regra own/adminOnly, monta UPDATE diretamente com os filtros fornecidos pelo chamador.
Alta	server/index.js:93-120; 642-657	**Leitura pública de perfis retorna campos de todas as empresas** O mecanismo de isolamento é filtro manual por user.id. Entretanto, appendReadScope libera integralmente jobs, freelancers, companies e public_profiles para qualquer usuário autenticado. A consulta final usa SELECT * sem projeção por campo.
Média	docker-compose.local.yml:9,23-24; server/seed.js:13,22; README.md:6,41,63	**Credenciais e defaults funcionais estão no repositório** O Compose contém senha do PostgreSQL e JWT_SECRET funcional de desenvolvimento; seed e documentação divulgam defaults de DATABASE_URL e senha das contas demo. O histórico Git também contém esses arquivos versionados.

Recomendações priorizadas

P1 - Remover o comportamento permissivo do CRUD: deny-by-default, allowlist de tabelas/campos/operações e ownership no SQL. P2 - Criar projeções públicas explícitas e retirar SELECT * de tabelas multiusuário. P3 - Eliminar defaults de segredo, exigir valores fortes em startup e rotacionar credenciais compartilhadas. P4 - Adicionar testes de autorização com dois usuários e testes de enumeração/IDOR para todos os handlers.

ISSUES PARA O GITHUB

ISSUE 1

```
--- ISSUE 1 ---
# [Segurança] POST de preferências aceita user_id controlado pelo cliente
Labels sugeridas: security, severity:alta

## Descrição
A policy de preferences habilita insert, mas o bloco de validações de POST cobre apenas jobs, applications,
conversations, messages e reviews. O INSERT usa todas as colunas enviadas no body, incluindo user_id, sem
substituí-lo pelo usuário autenticado.

## Evidência
server/index.js:71; 579-608
Trecho: preferences: { insert: true, update: false }; ... INSERT INTO ${table} (${cols.join(',')}) ... values do
body

## Impacto
Um usuário autenticado pode criar preferências associadas à conta de outro usuário, poluindo dados privados e
permitindo manipulação de estado em nome de terceiros. A rota dedicada /api/preferences é protegida, mas o app
mobile usa o CRUD genérico.

## Sugestão de correção
Remover preferences do CRUD genérico ou sobrescrever/validar user_id no servidor; preferir a rota dedicada já
existente e cobrir o app mobile com ela.

## Critérios de aceite
- [ ] POST com user_id de terceiro retorna 403
- [ ] user_id nunca é aceito do cliente para preferências
- [ ] Mobile usa endpoint dedicado com escopo por sessão

--- FIM ISSUE 1 ---
```

ISSUE 2

```
--- ISSUE 2 ---
# [Segurança] PATCH genérico altera objetos sem validação de posse
Labels sugeridas: security, severity:crítica

## Descrição
A rota /api/data/:table permite PATCH para qualquer tabela cujo policy.update seja undefined. O código valida
apenas a presença de eq.* e, para tabelas sem regra own/adminOnly, monta UPDATE diretamente com os filtros
fornecidos pelo chamador.

## Evidência
server/index.js:611-639
Trecho: const rule = policy.update; ... if (!filters.length) ...; UPDATE ${table} SET ... WHERE ${where}

## Impacto
Qualquer usuário autenticado pode alterar registros de jobs, applications, conversations, messages ou reviews
informando o ID de outro objeto. O risco inclui mudança de status, vínculos e conteúdo de terceiros.

## Sugestão de correção
Adotar deny-by-default: toda tabela deve declarar operações, campos e regra de ownership; rejeitar PATCH para
tabelas sem política explícita e aplicar a posse na própria cláusula WHERE.

## Critérios de aceite
- [ ] PATCH sem política explícita retorna 403
- [ ] A posse é aplicada no SQL, não apenas no frontend
- [ ] Testes cobrem cada tabela e IDs de terceiros

--- FIM ISSUE 2 ---
```

ISSUE 3

```
--- ISSUE 3 ---
# [Segurança] Leitura pública de perfis retorna campos de todas as empresas
Labels sugeridas: security, severity:alta
```

```
## Descrição
O mecanismo de isolamento é filtro manual por user.id. Entretanto, appendReadScope libera integralmente jobs,
freelancers, companies e public_profiles para qualquer usuário autenticado. A consulta final usa SELECT * sem
projeção por campo.

## Evidência
server/index.js:93-120; 642-657
Trecho: if (privilegedRead || ["jobs", "freelancers", "companies", "public_profiles"].includes(table)) return; ...
SELECT * FROM ${table}

## Impacto
Um usuário autenticado consegue enumerar perfis de empresas e receber campos internos como cnpj, além de outros
atributos que sejam adicionados ao schema. Não há tenant/workspace; a exposição depende apenas de possuir uma
sessão válida.

## Sugestão de correção
Separar dados públicos de dados privados, selecionar allowlists de colunas e proteger identificadores
empresariais. Não usar SELECT * para tabelas expostas a múltiplos papéis.

## Critérios de aceite
- [ ] Leituras públicas retornam somente campos públicos
- [ ] CNPJ e atributos internos não aparecem para terceiros
- [ ] Testes de enumeração entre dois usuários passam

--- FIM ISSUE 3 ---
```

ISSUE 4

```
--- ISSUE 4 ---
# [Segurança] Credenciais e defaults funcionais estão no repositório
Labels sugeridas: security, severity:média

## Descrição
O Compose contém senha do PostgreSQL e JWT_SECRET funcional de desenvolvimento; seed e documentação divulgam
defaults de DATABASE_URL e senha das contas demo. O histórico Git também contém esses arquivos versionados.

## Evidência
docker-compose.local.yml:9,23-24; server/seed.js:13,22; README.md:6,41,63
Trecho: POSTGRES_PASSWORD: trampo_local_dev; JWT_SECRET: trampo_local_jwt_dev_change_me; process.env.SEED_PASSWORD
|| "trampo123"

## Impacto
Se o Compose ou seed for reutilizado fora de um ambiente isolado, as credenciais previsíveis permitem acesso ao
banco ou emissão/aceitação de tokens com segredo conhecido. As contas demo são acessíveis por qualquer pessoa com
os dados documentados.

## Sugestão de correção
Mover valores para ambiente local não versionado, exigir segredos explícitos e fortes fora de desenvolvimento,
abortar com defaults conhecidos e rotacionar credenciais já compartilhadas.

## Critérios de aceite
- [ ] Nenhum segredo funcional fica em Compose, seed ou docs
- [ ] Startup rejeita defaults conhecidos fora de local
- [ ] Credenciais já divulgadas são rotacionadas

--- FIM ISSUE 4 ---
```